universität wien

MULTIOMICS DATA SCIENCE - CONTROL STRUCTURES

WHAT WE KNOW SO FAR

- We learned about data types and how to operate with them
- We stored information in variables and used / manipulated them
- Variables can be compared with each other
- Strings can be manipulated in different ways
- First programs / computations had a linear progress

CONTROL STRUCTURES

- Processors are designed to to execute more than one trivial order of commands
- If we define a program as a list of elemental commands, one can tell the processor to jump at a specific position
- Commands in between a jump will be ignored
- A backwards jump causes that commands will be repeatedly executed

CONTROL STRUCTURES

- Several control structures

- Branching structures:
  - if / then / else
  - switch

- loops
  - while
  - do / while
  - for

- Error handling
  - try / except

CONTROL STRUCTURES

- Several control structures

- Branching structures:
  - if / then / else
  - ~~switch~~ ← not supported in Python

- loops
  - while
  - ~~do / while~~ ← not supported in Python
  - for

- Error handling
  - try / except

IF / THEN / ELSE

- Current program process:
  1. command
  2. command
  3. command
  4. command
  ...

- program process very “boring“ and static

- It would be of interest, if programs would execute different parts of code depentent of certain conditions

CONDITIONS

- Conditions are statements that return either a `True` or `False` value
- We already know this statement kennen als comparison operators, e.g.

```
x > 10
```

  return a value, which is either `True` or `False`

- With if / then / else control structures it is possible to check conditions execute code or not

IF / THEN / ELSE CONTROL STRUCTURE

- The structure looks like that:

```
if condition:
    command
    command
```

- Or directly to execute in Python:

```
if x > 10:
    print("The variable x is bigger than 10")
```

INDENTATION

- Caution: brackets are used in other programming languages to indicate which commands belong to the `if` / `else` block

```
if (x > 10){
    y = 2;
    x += 3;
}
```

C++ / Java code

- In Python, the condition doesn‘t need (but may) be written in round brackets ()

- Python **doesn‘t** include curly brackets {} to determine the then / else blocks

INDENTATION

- Instead, the beginning of a `then` block starts after a colon :

```
if x > 10:
␣␣␣␣y = 2
␣␣␣␣x += 3
```

Python code

- Here, the sign ␣ stands for the space key

- After the `if` command, the next command must have at least one space more than the `if` command

- Now, all following commands must have the same indent depth as the first command in the corresponding block

universität wien

ELSE BLOCK

- Code can be executed if the condition is not satisfied, by using the `else` command:

```
if x > 10:
    print("x is at least 11")
else:
    print("x is at most 10")
```

- The `else` command must have again the same indent depth as its `if` command

- To start the next block, a colon : is again written after the `else` command

SEVERAL CONDITIONS

- Several different conditions can be put in a row with a `elif` command:

```
if age > 99:
    print("You are too old!")
elif age < 5:
    print("You are too young!")
else:
    print("You have permission to play the game!")
```

- As soon as the first condition is satisfied, all following conditions will be skipped

- Please consider the indent depth

NESTED CONTROL STRUCTURES

- You can nest several control structures into each other:

```
if x > 40 and y < 20:
    y += 40
    if z == 20:
        print("Case 1 occurs")
    else:
        print("Case 2 occurs")
else:
    x -= 100
```

- The complete (inner) blue `if` control structure is only processed, if the outer `if` condition is satisfied

NESTED CONTROL STRUCTURES

```
if x > 40:
    if y < 20:
        print("Code is executed")
```

has the same effect as:

```
if x > 40 and y < 20:
    print("Code is executed")
```

- Both conditions must be satisfied to execute the inner command

LOOPS

- If statements are designed to skip code blocks
- But we also want to execute code several times, that is jump back
- Intuitively: if I always jump back from a certain point, the program will never terminate
- There has to be a control mechanism to prevent this situation
- Idea: only jump back, when condition is satisfied

WHILE LOOP

- `While` loops have the same syntax (code setup) as `if` conditions:
  - the `while` command
  - a condition
  - followed by a :
  - and indented commands

  } loop head

  loop body

```
x = 1
while x < 10:
    print(x)
    x += 1
```

- The commands in the loop are executed as long as the condition is satisfied

universität wien

INFINITE LOOPS

- Caution, this structure may cause loops, that never terminate, e.g.:

```
x, y = 1, 1
while y < 10:
    print(x)
    x += 1
```

- In this case, the condition is dependent on `y`, but `y` is never updated
- Therefore, the examination of the condition doesn‘t change, `y < 10` remains always `True`
- And your computer runs until infinity and beyond ;-)

EXAMPLE – LITTLE GAUSS

- Exercise: compute the sum of the first *n* positive numbers, either with a `while` loop or (for cross checking) with the little Gauss

- Example for the sum of the first 10 numbers:

```
the_sum, intermediate, end_value = 0, 1, 10
while intermediate <= end_value:
    the_sum += intermediate
    intermediate += 1
print(the_sum, end_value * (end_value + 1) // 2)
```

- Caution: `sum` is a builtin function (**not** a keyword) in Python, and therefore it isn‘t recommendable to use it as a variable name although it is possible

FOR LOOPS

- The `for` loops use a slightly different concept than `while` loops

- These loops execute the loop body for all elements of a socalled “collection“ obejct

- Collection objects can be of different type:
  - variables, that contain more than one value (next week)
  - funktions, that return several values (in two weeks)

FOR LOOPS

```
for x in range(10):
    print(x)
```

- `For` loops contain a loop head with a collection object
- During each loop, the next element from the collection object is assigned to the variable `x`
- You almost can read the code naturally: for each object `x` within the collection from 0 to 10 (exclusively)
- With the loop body, you can overwrite `x`, but at the beginning of the next loop `x` will be assigned to the next element again

THE RANGE FUNCTION

- The Range function is a function, which yields whole numbers (integers) in their order within a defined range
- Here, you can again define a step value, similar to the slicing (last lecture)
- By default, the range always starts with 0 and the step value is 1 if not defined otherwise
- The end value of the range is always exclusive

THE RANGE FUNCTION - EXAMPLES

- `range(10)`
  output: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

- `range(3, 8)`
  output: 3, 4, 5, 6, 7

- `range(2, 20, 3)`
  output: 2, 5, 8, 11, 14, 17

- `range(10, 4, -1)`
  output: 10, 9, 8, 7, 6, 5

THE RANGE FUNCTION

- In generel, three different intput types:
  - `range(end_value*)`
  - `range(start_value, end_value*)`
  - `range(start_value, end_value*, step_value)`
- In contrast to slicing, this is a function, and parameters in function are seperated with a comma ,

* End values of a range are in Python always exclusive

universität wien

LITTLE GAUSS – ROUND 2

- We can also compute the sum of the first *n* numbers with `for` loops

```
the_sum end_value = 0, 10
for intermediate in range(end_value + 1):
    the_sum += intermediate
print(the_sum)
```

- If we want, that the end value will be included, we need to increment it by `+1` in the range function

LITTLE GAUSS – ROUND 3

- If collection objects only contain numbers, we can compute their sum with the sum function:

```
end_value = 10
the_sum = sum(range(end_value + 1))
print(the_sum)
```

- We learned now four methods to compute the sum of the first *n* positive numbers

- Some of these methods are „ more efficient“ or „better“ than the other

THE BREAK COMMAND

- It is possible to interrupt a loop early with the `break` command, for example:

```
for x in range(5):
    for y in range(5):
        if x + y >= 5: break
        print(y, end = " ")
    print()    # new line
```

```
Output:
0 1 2 3 4
0 1 2 3
0 1 2
0 1
0
```

- The `break` command only interrupts the most inner loop in which it is
- After executing the `break` command, the following code won‘t be executed anymore

universität wien

FOR LOOP ON STRINGS

- We recall, that strings are collections of characters, there it is possible to use strings in the `for` loop head:

```
cnt, my_string = 0, "Hello World"
for c in my_string:
    if c == "l":
        cnt += 1
print(cnt, my_string.count("l"))
```

- In this example, each character from the variable `my_string` will be assigned to variable `c` in each loop

- The code counts the occurrences of the letter “l“ in the word

INLINE IF STATEMENT

- Often we have the situation that we have to assign a conditional value to variable, e.g.:

```
if value > 100:
    my_var = 1
else:
    my_var = -1
```

- A neet solution are „inline“ statements with the following structure:

[variable] = [value on true condition] `if` [condition] `else` [value on false condition]

```
my_var = 1 if value > 100 else -1
```

EXCEPTION HANDLING

- It often happens that the execution of commands results in an error

- Possible reasons are:
  - Incorrect operations on variables (e.g. adding auf Strings)
  - Unexpected situation (write a file on a full disk)
  - Domainn error, e.g. division by 0 or logarithm of a negative number

- Without an exception handling, the program terminates immediately on the first error

- Many programming languages (such as Python) offer a control structures for exception handling and to avoid that the program „crashes“

EXCEPTION HANDLING

- Catching exceptions happens in `try` / `except` blocks:

```
try:
    divident, divisor = 5, 0
    quotient = divident / divisor
except:
    print("An error happened!")
print("My code will go on!")
```

- A possible error in the `try` block, will be caught
- This error can be handled in the `except` block
- If no error happens, the `except` block will be skipped

EARLY PROGRAM TERMINATION

- To terminate the program early, you can use the the `exit()` function:

```
age = 35
if age < 5 or 99 < age:
    print("You shall not pass!")
    exit()
print("You may pass")
```

- Inernally, the program jumps to its end

EARLY PROGRAM TERMINATION

- To terminate the program early, you can use the the `exit()` function:

```
age = 35
if age < 5 or 99 < age:
    print("You shall not pass!")
    exit()
print("You may pass")
```

- Inernally, the program jumps to its end

Source: https://www.pinterest.com/pin/564357397036631811/